

Guia rapida de React Native con Expo

JavaScript puro - componentes, estilos y patrones practicos

1. Que es Expo en pocas palabras

Expo es una plataforma y framework sobre React Native que permite crear apps universales para Android, iOS y web usando JavaScript y React. Para iniciar un proyecto nuevo puedes usar create-expo-app, que genera una base lista para desarrollar.

```
npx create-expo-app@latest mi-app
cd mi-app
npx expo start
```

2. Componentes mas usados

Componente	Uso principal	Ejemplo tipico
View	Contenedor base para layout.	Pantallas, cards, filas, columnas.
Text	Mostrar texto.	Titulos, parrafos, labels, mensajes.
Image	Mostrar imagenes.	Avatares, banners, logos.
ScrollView	Scroll para contenido pequeno o mediano.	Formularios cortos, pantallas informativas.
FlatList	Lista optimizada para muchos elementos.	Feeds, productos, chats, usuarios.
SectionList	Lista por secciones.	Contactos por letra, categorias.
TextInput	Entrada de texto.	Login, busqueda, formularios.
Pressable	Zona tactil moderna y flexible.	Botones personalizados, cards clicables.
TouchableOpacity	Boton tactil con opacidad.	Acciones simples o componentes legacy.
Button	Boton nativo basico.	Prototipos y acciones rapidas.
SafeAreaView	Evita zonas del sistema.	Pantallas con notch o status bar.
StatusBar	Control de barra superior.	Modo claro/oscurito de la barra.
ActivityIndicator	Indicador de carga.	Pantallas loading, botones con espera.
Modal	Ventana flotante nativa.	Confirmaciones, menus, formularios.
KeyboardAvoidingView	Evita que el teclado tape inputs.	Login, registro, chats.
Switch	Activar/desactivar.	Preferencias y configuraciones.
Alert	Alerta nativa simple.	Confirmar eliminacion o mostrar error.
Platform	Detectar iOS, Android o web.	Estilos y logica por plataforma.

3. Ejemplo base en JavaScript puro

```
import { StatusBar } from 'expo-status-bar';
import {
  View,
  Text,
  StyleSheet,
  Image,
  Pressable,
  SafeAreaView,
} from 'react-native';

export default function App() {
  return (
    <SafeAreaView style={styles.screen}>
      <View style={styles.card}>
        <Image
          source={{ uri: 'https://picsum.photos/300' }}
          style={styles.image}
        />

        <Text style={styles.title}>Hola Expo</Text>
      </View>
    </SafeAreaView>
  );
}
```

```

    <Text style={styles.description}>
      Esta tarjeta esta escrita con JavaScript puro.
    </Text>

    <Pressable style={styles.button} onPress={() => console.log('click')}>
      <Text style={styles.buttonText}>Continuar</Text>
    </Pressable>
  </View>
  <StatusBar style="auto" />
</SafeAreaView>
);
}

const styles = StyleSheet.create({
  screen: {
    flex: 1,
    backgroundColor: '#F5F7FA',
    padding: 20,
    justifyContent: 'center',
  },
  card: {
    backgroundColor: 'FFFFFF',
    borderRadius: 20,
    padding: 20,
    gap: 12,
    shadowColor: '000',
    shadowOpacity: 0.1,
    shadowRadius: 12,
    elevation: 4,
  },
  image: {
    width: '100%',
    height: 180,
    borderRadius: 16,
  },
  title: {
    fontSize: 24,
    fontWeight: '700',
    color: '#111827',
  },
  description: {
    fontSize: 16,
    color: '#6B7280',
    lineHeight: 22,
  },
  button: {
    backgroundColor: '#2563EB',
    paddingVertical: 14,
    borderRadius: 12,
    alignItems: 'center',
    marginTop: 8,
  },
  buttonText: {
    color: 'FFFFFF',
    fontWeight: '700',
    fontSize: 16,
  },
});

```

4. Mini guia de estilos

4.1 Usa StyleSheet.create

Mantiene los estilos ordenados, reutilizables y fuera del render. Es ideal para componentes medianos y grandes.

```
import { StyleSheet } from 'react-native';

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    backgroundColor: '#FFFFFF',
  },
});
```

4.2 Flexbox es la base del layout

Propiedad	Uso
flex: 1	Ocupa todo el espacio disponible.
flexDirection	Define fila o columna.
justifyContent	Alinea en el eje principal.
alignItems	Alinea en el eje secundario.
gap	Espacio entre hijos.
padding / margin	Espacio interno y externo.

```
const styles = StyleSheet.create({
  row: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    gap: 12,
  },
});
```

4.3 Crea un tema central

Centralizar colores, espacios y radios evita inconsistencias visuales y hace mas facil cambiar la identidad de la app.

```
export const theme = {
  colors: {
    primary: '#2563EB',
    background: '#F5F7FA',
    text: '#111827',
    muted: '#6B7280',
    white: '#FFFFFF',
  },
  spacing: {
    xs: 4,
    sm: 8,
    md: 16,
    lg: 24,
    xl: 32,
  },
  radius: {
    sm: 8,
    md: 12,
    lg: 20,
  },
};
```

4.4 Estilos condicionales

```
<Pressable style={[styles.button, isActive && styles.buttonActive]}>
  <Text style={styles.buttonText}>Guardar</Text>
</Pressable>

const styles = StyleSheet.create({
  button: {
    backgroundColor: '#E5E7EB',
    padding: 14,
    borderRadius: 12,
  },
  buttonActive: {
    backgroundColor: '#2563EB',
  },
});
```

4.5 Diferencias por plataforma

```
import { Platform, StyleSheet } from 'react-native';

const styles = StyleSheet.create({
  card: {
    padding: 20,
    ...Platform.select({
      ios: {
        shadowColor: '#000',
        shadowOpacity: 0.15,
        shadowRadius: 10,
      },
      android: {
        elevation: 5,
      },
    }),
  },
});
```

5. Listas: ScrollView vs FlatList

- Usa ScrollView cuando el contenido es limitado y no hay cientos de elementos.
- Usa FlatList cuando tienes listas largas o datos dinámicos.
- FlatList renderiza elementos de manera optimizada y evita cargar toda la lista al mismo tiempo.

```
const users = [
  { id: '1', name: 'Ana' },
  { id: '2', name: 'Luis' },
];

<FlatList
  data={users}
  keyExtractor={({item}) => item.id}
  renderItem={({ item }) => (
    <Text style={styles.item}>{item.name}</Text>
  )}
/>
```

6. Estructura recomendada

```
src/
  components/
    AppButton.js
    Card.js
    Input.js
  screens/
```

```
    HomeScreen.js
    LoginScreen.js
  theme/
    colors.js
    spacing.js
    typography.js
  hooks/
  utils/
App.js
```

7. Librerías comunes con Expo

Librería	Uso
expo-router	Navegación basada en archivos.
expo-image	Imágenes optimizadas.
expo-font	Fuentes personalizadas.
expo-linear-gradient	Gradientes.
react-native-reanimated	Animaciones avanzadas.
react-native-gesture-handler	Gestos complejos.
react-native-safe-area-context	Safe areas avanzado.
react-hook-form	Formularios.
zod	Validación de datos.
zustand	Estado global simple.
nativewind	Estilos tipo Tailwind.

8. Reglas rápidas

- View para estructura, Text para texto, Image para imágenes.
- Pressable para interacciones personalizadas.
- TextInput para formularios y búsquedas.
- ScrollView para contenido pequeño; FlatList para listas grandes.
- Mantén colores y espaciados en un theme.
- Evita estilos inline repetidos en muchas pantallas.
- Separa componentes reutilizables: botones, cards, inputs y headers.

9. Fuentes oficiales consultadas

- React Native - Core Components and APIs: <https://reactnative.dev/docs/components-and-apis>
- React Native - StyleSheet: <https://reactnative.dev/docs/stylesheet>
- React Native - ScrollView: <https://reactnative.dev/docs/scrollview>
- Expo Documentation: <https://docs.expo.dev/>
- Expo create-expo-app: <https://docs.expo.dev/more/create-expo/>